



# Guía del Estudiante — Pruebas Unitarias, JaCoCo y Pipeline CI

Asignatura DevOps | Actividad 2.3 — Asegurando calidad en nuestro código

DUOC UC · DevOps 003D

JUnit 5 · Mockito · JaCoCo

GitHub Actions

¿Qué vas a lograr? Escribir pruebas unitarias para la API REST de Estudiantes, medir la cobertura de código con JaCoCo y automatizar todo en un pipeline de GitHub Actions que guarde el reporte de cobertura como artefacto. No necesitas levantar contenedores para ejecutar los tests — Maven los corre directamente en tu máquina.



## Paso 0 — Verifica tu entorno local

Antes de comenzar, confirma que tienes las herramientas necesarias:

| Herramienta             | ¿Cómo verificar?                                              | Versión mínima |
|-------------------------|---------------------------------------------------------------|----------------|
| Java JDK                | <code>java -version</code>                                    | 17             |
| Maven                   | <code>mvn -version</code>                                     | 3.8+           |
| Git                     | <code>git --version</code>                                    | cualquiera     |
| Docker Desktop o Podman | <code>docker --version</code> / <code>podman --version</code> | cualquiera     |

⚠ Para los tests unitarios solo necesitas Java y Maven. Docker/Podman se usa más adelante para el pipeline completo y para levantar el sistema (como en la actividad anterior).



## Paso 1 — Obtén el proyecto

Clona o descomprime el proyecto en tu máquina.

```
bdget/
├── .github/
│   └── workflows/
│       └── main.yml      ← pipeline de GitHub Actions (lo modificarás aquí)
├── src/
│   ├── main/java/...    ← código fuente de la aplicación
│   └── test/java/...    ← pruebas unitarias (aquí trabajarás)
├── pom.xml              ← dependencias y configuración de JaCoCo
└── Dockerfile
```

Abre una terminal y entra a la raíz del proyecto:

```
cd bdget
```

⚠ Todos los comandos Maven de esta guía se ejecutan desde la raíz del proyecto (donde está `pom.xml`).

## 🔍 Paso 2 — Entiende la estructura de pruebas existente

El proyecto ya tiene tres clases de test. Ábrelas y lee cada una antes de ejecutar nada.

```
src/test/java/com/example/bdget/
├── BdgetApplicationTests.java      ← test de arranque del contexto
├── model/
│   └── StudentModelTest.java      ← prueba del modelo (getters/setters)
├── service/
│   └── StudentServiceImplTest.java ← prueba del servicio con Mockito
└── controller/
    └── StudentControllerTest.java ← prueba del controlador con MockMvc
```

### ¿Qué tipo de prueba es cada una?

| Clase                               | Tipo                      | Herramienta       | ¿Levanta contexto Spring?              |
|-------------------------------------|---------------------------|-------------------|----------------------------------------|
| <code>StudentModelTest</code>       | Prueba unitaria pura      | JUnit 5           | ❌ No                                   |
| <code>StudentServiceImplTest</code> | Prueba unitaria con mocks | JUnit 5 + Mockito | ❌ No                                   |
| <code>StudentControllerTest</code>  | Prueba de capa web        | MockMvc + Mockito | ✅ Parcial ( <code>@WebMvcTest</code> ) |

💡 **Concepto clave — Prueba unitaria vs. de integración:** Una prueba *unitaria* aísla un componente y reemplaza sus dependencias con *mocks*. Una prueba de *integración* levanta el sistema completo y conecta componentes reales. Esta actividad se enfoca en pruebas unitarias para detectar errores rápido y sin base de datos.

## 🔍 Paso 3 — Lee el pom.xml: JaCoCo ya está configurado

Abre `pom.xml` y localiza el plugin de JaCoCo:

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.10</version>
  <executions>
    <!-- Instrumenta el bytecode ANTES de ejecutar tests -->
    <execution>
      <goals>
        <goal>prepare-agent</goal>    <!-- fase: initialize -->
      </goals>
    </execution>
    <!-- Genera el reporte HTML DESPUÉS de ejecutar tests -->
    <execution>
      <id>report</id>
      <phase>test</phase>            <!-- se dispara al final de mvn test -->
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

### ¿Qué hace JaCoCo?

JaCoCo instrumenta el bytecode compilado de tu aplicación. Cuando los tests se ejecutan, registra qué líneas, ramas e instrucciones fueron alcanzadas. Al terminar, genera un reporte HTML visual en `target/site/jacoco/index.html`.

## Paso 4 — Ejecuta los tests localmente

### Ejecutar todos los tests y generar cobertura

```
mvn test
```

Maven ejecutará las 3 clases de test y, gracias a la configuración de JaCoCo en el `pom.xml`, generará el reporte automáticamente al finalizar.

Salida esperada (resumen final):

```
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.10:report (report) ---
[INFO] Loading execution data file ../target/jacoco.exec
[INFO] Analyzed bundle 'bdget' with 5 classes
[INFO] BUILD SUCCESS
```

 La primera vez tarda ~1-2 minutos porque Maven descarga dependencias. Las siguientes veces es casi instantáneo.

## Paso 5 — Abre el reporte de cobertura

El reporte HTML queda en: `target/site/jacoco/index.html`

Ábrelo con tu navegador:

### Con Docker Desktop o Podman (macOS)

```
open target/site/jacoco/index.html
```

### En Linux

```
xdg-open target/site/jacoco/index.html
```

### En Windows (PowerShell)

```
start target\site\jacoco\index.html
```

### ¿Qué verás en el reporte?

El reporte muestra una tabla con cada paquete y clase:

| Package                      | Missed | Coverage                                                                                 |
|------------------------------|--------|------------------------------------------------------------------------------------------|
| com.example.bdget.model      | 0      | 100%  |
| com.example.bdget.service    | 2      | 85%   |
| com.example.bdget.controller | 0      | 100%  |
| com.example.bdget.repository | N/A    | N/A (interface)                                                                          |

| Color en JaCoCo                                                                           | Significado                                                                         |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  Verde    | Línea/rama ejecutada por al menos un test                                           |
|  Rojo     | Línea/rama NO ejecutada — posible gap de cobertura                                  |
|  Amarillo | Rama parcialmente cubierta (ej: solo el <code>if</code> , no el <code>else</code> ) |

## Paso 6 — Analiza las pruebas existentes

### StudentModelTest.java — Prueba del modelo

```
@Test
void testGettersAndSetters() {
    Student student = new Student();
    student.setId(1L);
    student.setName("John");
    assertEquals(1L, student.getId()); // verifica getter id
    assertEquals("John", student.getName()); // verifica getter name
}
```

Esta prueba verifica que los getters y setters del modelo `Student` funcionen correctamente. Es una prueba unitaria pura: no usa Spring, no usa base de datos.

### StudentServiceImplTest.java — Prueba del servicio

```
@ExtendWith(MockitoExtension.class) // activa Mockito
class StudentServiceImplTest {

    @Mock
    private StudentRepository repository; // MOCK: simula la BD

    @InjectMocks
    private StudentServiceImpl service; // objeto real bajo prueba

    @Test
    void testUpdateStudentExists() {
        when(repository.existsById(1L)).thenReturn(true); // simula que el ID existe
        when(repository.save(student)).thenReturn(student);
        Student result = service.updateStudent(1L, student);
        assertEquals(student, result);
        verify(repository).save(student); // verifica que se llamó a save()
    }

    @Test
    void testUpdateStudentNotExists() {
        when(repository.existsById(1L)).thenReturn(false); // simula que NO existe
        assertNull(service.updateStudent(1L, student)); // debe retornar null
        verify(repository, never()).save(any()); // NO debe llamar a save()
    }
}
```

 **@Mock** vs **@InjectMocks**: **@Mock** crea un objeto falso que simula el comportamiento que tú definas. **@InjectMocks** crea el objeto real e inyecta automáticamente los mocks en él. Así probamos `StudentServiceImpl` sin tocar la base de datos.

```
@WebMvcTest(StudentController.class) // levanta SOLO la capa web
class StudentControllerTest {

    @Autowired
    private MockMvc mockMvc; // simula peticiones HTTP sin servidor real

    @MockBean
    private StudentService service; // mock del servicio (no se usa BD)

    @Test
    void testGetAllStudents() throws Exception {
        when(service.getAllStudents()).thenReturn(Arrays.asList(student));

        mockMvc.perform(get("/students")) // GET /students
            .andExpect(status().isOk()) // espera HTTP 200
            .andExpect(content().json(...)); // verifica el JSON
    }
}
```

💡 **@WebMvcTest** vs **@SpringBootTest**: **@WebMvcTest** carga solo los beans de la capa web (controladores, filtros). No levanta JPA ni la base de datos. Es más rápido y apropiado para pruebas unitarias del controlador.

### ✏ Paso 7 — Agrega tus propias pruebas

Para mejorar la cobertura, puedes agregar casos que el código existente no cubre. Por ejemplo, el caso en que `getStudentById` no encuentra el estudiante:

Abre `src/test/java/com/example/bdget/service/StudentServiceImplTest.java` y agrega:

```
@Test
void testGetStudentByIdNotFound() {
    when(repository.findById(99L)).thenReturn(Optional.empty());
    Optional<Student> result = service.getStudentById(99L);
    assertTrue(result.isEmpty());
}
```

O en `StudentControllerTest.java`, el caso de estudiante no encontrado (404):

```
@Test
void testGetStudentByIdNotFound() throws Exception {
    when(service.getStudentById(99L)).thenReturn(Optional.empty());
    mockMvc.perform(get("/students/99"))
        .andExpect(status().isNotFound());
}
```

Después de agregar tests, vuelve a ejecutar:

```
mvn test
```

Y abre nuevamente `target/site/jacoco/index.html` para ver cómo mejoró la cobertura.

### 🔄 Paso 8 — Lee el pipeline de GitHub Actions

Abre `.github/workflows/main.yml` y analiza cada paso:

```

name: CI – Build, Test y Push a DockerHub

on:
  push:
    branches: [main]      # se dispara al hacer push a main
  pull_request:
    branches: [main]      # también en pull requests

jobs:
  build-test-push:
    runs-on: ubuntu-latest

    services:
      mysql:
        # servicio MySQL para los tests
        image: mysql:8.0
        env:
          MYSQL_DATABASE: bdget_db
          MYSQL_USER: bdget_user
          MYSQL_PASSWORD: bdget_pass
        options: --health-cmd="mysqladmin ping ..." --health-retries=5

    steps:
      - uses: actions/checkout@v4          # 1. Descarga el código

      - uses: actions/setup-java@v4        # 2. Instala Java 17 + caché Maven
        with:
          java-version: '17'
          cache: maven

      - name: Ejecutar tests                # 3. Corre tests + genera jacoco.exec
        run: mvn test
        env:
          SPRING_DATASOURCE_URL: jdbc:mysql://localhost:3306/bdget_db?...

      - name: Generar reporte de cobertura  # 4. jacoco.exec → HTML
        run: mvn jacoco:report

      - name: Subir reporte de cobertura    # 5. Guarda HTML como artefacto
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/

      - run: mvn package -DskipTests        # 6. Compila el .jar
      - uses: docker/login-action@v3        # 7. Login en DockerHub
      - run: docker build -t usuario/bdget-app:latest . # 8. Build imagen
      - run: docker push usuario/bdget-app:latest      # 9. Push a DockerHub

```

| Paso                                                | ¿Para qué sirve?                                                               |
|-----------------------------------------------------|--------------------------------------------------------------------------------|
| <code>actions/checkout@v4</code>                    | Descarga el código del repositorio en el runner                                |
| <code>setup-java</code> + <code>cache: maven</code> | Instala Java 17 y reutiliza dependencias descargadas                           |
| <code>mvn test</code>                               | Corre todos los tests y genera <code>jacoco.exec</code>                        |
| <code>mvn jacoco:report</code>                      | Convierte <code>jacoco.exec</code> en HTML en <code>target/site/jacoco/</code> |
| <code>upload-artifact@v4</code>                     | Guarda el HTML como artefacto descargable en GitHub                            |
| <code>mvn package -DskipTests</code>                | Compila el <code>.jar</code> sin re-ejecutar tests                             |

## Paso 9 — Configura los Secrets en GitHub

El pipeline necesita credenciales de DockerHub guardadas como Secrets en GitHub.

1. Ve a tu repositorio en GitHub
2. Click en **Settings** → **Secrets and variables** → **Actions**
3. Click en **"New repository secret"** y agrega:

| Nombre del Secret               | Valor                                           |
|---------------------------------|-------------------------------------------------|
| <code>DOCKERHUB_USERNAME</code> | Tu nombre de usuario en DockerHub               |
| <code>DOCKERHUB_TOKEN</code>    | Token de acceso de DockerHub (no tu contraseña) |

### ¿Cómo crear un token de DockerHub?

1. Ingresa a `https://hub.docker.com`
2. Click en tu usuario → Account Settings → Security
3. Click en "New Access Token"
4. Ponle un nombre (ej: "github-actions") y copia el token generado
5. Pégallo en GitHub como el secret `DOCKERHUB_TOKEN`

⚠ Un token de acceso es más seguro que tu contraseña: puedes revocarlo sin cambiar tu contraseña. Nunca escribas credenciales directamente en el YAML del pipeline.

## Paso 10 — Ejecuta el pipeline en GitHub Actions

1. Asegúrate de tener todos los cambios commiteados:

```
git add .
git commit -m "feat: agrego pruebas unitarias y pipeline con JaCoCo"
git push origin main
```

2. Ve a tu repositorio en GitHub → pestaña **Actions**
3. Verás el workflow **"CI — Build, Test y Push a DockerHub"** ejecutándose
4. Click en el nombre del workflow para ver el detalle de cada paso

### ¿Qué verás en la ejecución?

|                                 |                              |
|---------------------------------|------------------------------|
| ✓ Checkout del repositorio      | ~5s                          |
| ✓ Configurar Java 17            | ~15s                         |
| ✓ Ejecutar tests                | ~60s (12 tests, 0 fallos)    |
| ✓ Generar reporte de cobertura  | ~5s                          |
| ✓ Subir reporte de cobertura    | ~5s → aparece en "Artifacts" |
| ✓ Build del proyecto            | ~30s                         |
| ✓ Login en DockerHub            | ~5s                          |
| ✓ Build de la imagen Docker     | ~120s                        |
| ✓ Push de la imagen a DockerHub | ~30s                         |

## Paso 11 — Descarga y analiza el artefacto de cobertura

Una vez que el pipeline termine exitosamente:

1. Ve a la pestaña **Actions** en GitHub

2. Click en la ejecución del workflow
3. Baja hasta la sección **Artifacts**
4. Descarga `jacoco-report`
5. Descomprime el `.zip` y abre `index.html` en tu navegador

Verás exactamente el mismo reporte que generaste localmente, pero guardado permanentemente en GitHub como evidencia de la ejecución del pipeline.

---

## Solución de problemas frecuentes

---

### Los tests fallan con error de conexión a la base de datos

Las pruebas unitarias no necesitan MySQL. Puedes ejecutar solo los tests unitarios:

```
mvn test -Dtest=StudentModelTest,StudentServiceImplTest
```

`mvn test` falla con "compilation error"

Verifica que estás usando Java 17:

```
java -version
# Debe mostrar: openjdk version "17.x.x"

# Si tienes múltiples versiones de Java (macOS):
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
mvn test
```

### El reporte JaCoCo no se genera

Si `target/site/jacoco/` no existe después de `mvn test`, fuerza la generación:

```
mvn jacoco:report
```

### El pipeline falla en "Push de la imagen a DockerHub"

Verifica que los secrets `DOCKERHUB_USERNAME` y `DOCKERHUB_TOKEN` estén configurados en GitHub (Settings → Secrets → Actions).

### El pipeline falla en "Ejecutar tests" por MySQL

El pipeline usa un servicio MySQL de GitHub Actions. Si los tests unitarios no necesitan base de datos, puedes omitir el bloque `services:` del YAML o asegúrate de que la URL en `env:` apunte a `localhost:3306`.

---

## Resumen de comandos Maven

| Acción                               | Comando                                         |
|--------------------------------------|-------------------------------------------------|
| Ejecutar todos los tests + cobertura | <code>mvn test</code>                           |
| Ejecutar solo una clase de test      | <code>mvn test -Dtest=NombreClase</code>        |
| Generar reporte JaCoCo manualmente   | <code>mvn jacoco:report</code>                  |
| Compilar sin tests                   | <code>mvn package -DskipTests</code>            |
| Limpiar archivos compilados          | <code>mvn clean</code>                          |
| Todo desde cero                      | <code>mvn clean test</code>                     |
| Ver reporte local (macOS)            | <code>open target/site/jacoco/index.html</code> |

## Lista de verificación — entregables

Al terminar la actividad debes tener evidencia de cada punto:

- ☐ Captura de `mvn test` con **0 fallos** y el número de tests ejecutados
- ☐ Captura del reporte `index.html` de JaCoCo abierto en el navegador (porcentaje de cobertura visible)
- ☐ Captura del reporte de JaCoCo mostrando líneas cubiertas (verde) y no cubiertas (rojo) en al menos una clase
- ☐ Captura del pipeline en GitHub Actions con **todos los pasos en verde** 
- ☐ Captura de la sección **Artifacts** en GitHub Actions mostrando el artefacto `jacoco-report` descargable
- ☐ Captura del reporte de cobertura descargado desde el artefacto de GitHub Actions